

CPSC 250

Fibonacci Sequence and Algorithm Complexity

Edward Brash

1 The Fibonacci Sequence

The Fibonacci sequence is defined by:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$$

The sequence begins with:

$$f_0 = 0$$

$$f_1 = 1$$

and every later term is the sum of the previous two:

$$f_n = f_{n-1} + f_{n-2}$$

2 Recursive Fibonacci

The recursive definition naturally suggests a recursive function.

```
def fibonacci(n):  
    if n == 0:  
        return 0  
  
    elif n == 1:  
        return 1  
  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

3 Testing the Recursive Function

$$fibonacci(0) = 0$$

$$fibonacci(1) = 1$$

$$fibonacci(2) = fibonacci(1) + fibonacci(0) = 1$$

$$fibonacci(3) = fibonacci(2) + fibonacci(1) = 2$$

The recursive implementation works correctly.

4 The Computational Problem

Although recursive Fibonacci is elegant, it is extremely inefficient.

Why?

Because the function repeatedly recalculates the same values.

For example:

$fibonacci(5)$

requires:

$fibonacci(4)$

and:

$fibonacci(3)$

But:

$fibonacci(4)$

also requires another computation of:

$fibonacci(3)$

and so on.

5 Function Call Explosion

The number of recursive calls grows very rapidly.

Approximate call counts:

n	Number of Calls
0	1
1	1
2	3
3	4
4	7
5	11
6	18
7	29
8	47
9	76

By:

$$n = 30$$

the number of recursive calls is enormous.

6 Complexity of Recursive Fibonacci

The recursive algorithm has approximately exponential complexity:

$$O(e^n)$$

This is extremely bad scaling behavior.
As n increases, runtime grows explosively.

7 Non-Recursive Fibonacci

We can do much better using iteration.

```
def fibonacci(n):  
  
    if n == 0:  
        return 0  
  
    elif n == 1:  
        return 1  
  
    else:  
  
        t1 = 0  
        t2 = 1  
  
        for i in range(2, n + 1):  
  
            t3 = t2 + t1  
  
            t1 = t2  
            t2 = t3  
  
        return t3
```

8 Complexity of Iterative Fibonacci

The iterative version only performs one loop over the data.
Its complexity is:

$$O(n)$$

This is dramatically better than exponential growth.

9 The Golden Ratio

The ratio of successive Fibonacci numbers approaches a constant:

$$\phi \approx 1.618033988\dots$$

called the golden ratio.
Using:

$$f_n = f_{n-1} + f_{n-2}$$

and dividing by f_{n-1} :

$$\frac{f_n}{f_{n-1}} = 1 + \frac{f_{n-2}}{f_{n-1}}$$

As n becomes large:

$$\phi = 1 + \frac{1}{\phi}$$

This gives:

$$\phi^2 - \phi - 1 = 0$$

Using the quadratic formula:

$$\phi = \frac{1 + \sqrt{5}}{2}$$

10 Closed-Form Fibonacci Formula

Binet's formula gives a direct expression for Fibonacci numbers:

$$f_n = \frac{\phi^n - \left(-\frac{1}{\phi}\right)^n}{\sqrt{5}}$$

This suggests a constant-time Fibonacci algorithm.

11 Formula-Based Fibonacci

```
import math

phi = (1.0 + math.sqrt(5.0)) / 2.0

def fibonacci(n):

    return int(
        (phi**n - (-1.0 / phi)**n)
        / math.sqrt(5.0)
    )
```

12 Floating Point Precision Problems

For large n , floating-point precision errors appear.

Why?

Because numbers like:

$$\sqrt{5}$$

and:

ϕ

cannot be represented exactly in binary floating-point form.
Eventually the rounding errors accumulate.

13 Using the Decimal Module

Python's `decimal` module allows arbitrary precision arithmetic.

```
import decimal as dec  
  
dec.getcontext().prec = 50
```

Now calculations can be performed with much higher precision.

14 Summary of Fibonacci Approaches

Method	Complexity	Notes
Recursive	$O(e^n)$	Elegant but slow
Iterative	$O(n)$	Efficient and practical
Formula-Based	$O(1)$	Fast but precision-sensitive

15 Key Takeaways

- Recursive algorithms can be elegant but inefficient.
- Complexity describes how runtime scales with input size.
- Exponential complexity becomes unusable quickly.
- Iterative solutions are often dramatically faster.
- Floating-point precision matters in numerical computing.
- Python's `decimal` module supports high-precision arithmetic.